Produto matriz-vetor com MPI

Matheus Marinho 29 de setembro de 2026

O que o programa faz

Conta: `y[i] = soma(A[i,j] × x[j])`, para `j` de `0` até `N-1`. Cada linha produz um elemento de `y` e pode ser calculada separadamente. A matriz fica em um vetor de memória, com as colunas de cada linha em sequência.

Divisão do trabalho: o processo 0 cria `A` e `x`. Com `P` processos, cada um recebe `ceil(M/P)` linhas. Se sobram posições, `calloc` deixa essas linhas em zero; o processo 0 usa apenas os primeiros `M` resultados. Isso permite usar `MPI_Scatter` e `MPI_Gather`, que enviam a mesma quantidade a cada processo.

chamada	papel no programa
`MPI_Comm_rank` / `MPI_Comm_size`	Identificam o processo e o total de processos.
`MPI_Scatter`	Reparte as linhas de `A` entre os processos.
`MPI_Bcast`	Entrega `x` inteiro a todos: toda linha precisa do vetor completo.
Laços `i` e `j`	`i` percorre as linhas locais; `j` faz a soma dos produtos.
`MPI_Gather`	Reúne as partes de `y` em ordem no processo 0.

Teste de correção: com `M=5`, `N=3`, a primeira linha é `1,2,3`, então `y[0]=6`; a última é `5,6,7`, então `y[4]=18`. Os dois valores saíram iguais com 1, 2 e 8 processos. O caso de 8 processos também testa linhas extras.

Tempos medidos

Ambiente: NPAD, nó `r2n09`, partição `amd-512`, um nó exclusivo; GCC 8.5.0, Open MPI 4.1.6, compilação com `-O2 -Wall -Wextra -std=c11`. Para cada tamanho e número de processos, foram feitas cinco execuções; a tabela mostra a **mediana**, em segundos. `MPI_Barrier` alinha o início; `MPI_Wtime` mede de antes do `Scatter` até depois do `Gather` no processo 0. A criação de `A` e `x` fica fora desse intervalo.

Matriz	1 processo	2 processos	4 processos	8 processos
2048 × 2048	0,008543	0,014674	0,021900	0,023500
4096 × 4096	0,033235	0,040298	0,040465	0,043019
8192 × 8192	0,131454	**0,120667**	0,132492	0,126684

Leitura: em 2048², 8 processos levam cerca de 2,75 vezes o tempo de 1. Em 8192², 2 processos dão ganho pequeno: `T(1)/T(2) ≈ 1,09`. Os tempos de 4 e 8 processos nessa matriz são próximos do tempo com 1. Mais processos reduzem as linhas calculadas por cada um, mas distribuir `A` e reunir `y` também custa tempo. Essa é uma explicação provável para o ganho limitado; o programa não mediu cada fase separadamente.

Para defender: `rank` é o número de cada processo; as três chamadas de comunicação são *coletivas*, portanto todos os processos as executam na mesma ordem. *Speedup* compara `T(1)/T(P)` para o mesmo tamanho. Aqui, `T(1)` já inclui as chamadas MPI com um processo, não é uma versão sequencial separada.

Código usado

matriz_vetor_mpi.c · 83 linhas

```c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int processo, processos;
    MPI_Comm_rank(MPI_COMM_WORLD, &processo);
    MPI_Comm_size(MPI_COMM_WORLD, &processos);

    if (argc != 3 || atoi(argv[1]) <= 0 || atoi(argv[2]) <= 0) {
        if (processo == 0) {
            fprintf(stderr, "Uso: %s M N (ambos positivos)\n", argv[0]);
        }
        MPI_Finalize();
        return 1;
    }

    int m = atoi(argv[1]);
    int n = atoi(argv[2]);
    int linhas_locais = (m + processos - 1) / processos;
    int linhas_distribuidas = linhas_locais * processos;

    double *a = NULL;
    double *y = NULL;
    double *x = malloc((size_t)n * sizeof(double));
    double *a_local = malloc((size_t)linhas_locais * n * sizeof(double));
    double *y_local = malloc((size_t)linhas_locais * sizeof(double));

    if (processo == 0) {
        a = calloc((size_t)linhas_distribuidas * n, sizeof(double));
        y = malloc((size_t)linhas_distribuidas * sizeof(double));
    }
    if (!x || !a_local || !y_local || (processo == 0 && (!a || !y))) {
        fprintf(stderr, "Processo %d: memoria insuficiente\n", processo);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (processo == 0) {
        for (int j = 0; j < n; j++) {
            x[j] = 1.0;
        }
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                a[(size_t)i * n + j] = (double)((i + j) % 10 + 1);
            }
        }
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double inicio = MPI_Wtime();

    MPI_Scatter(a, linhas_locais * n, MPI_DOUBLE,
                a_local, linhas_locais * n, MPI_DOUBLE,
                0, MPI_COMM_WORLD);
    MPI_Bcast(x, n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    for (int i = 0; i < linhas_locais; i++) {
        double soma = 0.0;
        for (int j = 0; j < n; j++) {
            soma += a_local[(size_t)i * n + j] * x[j];
        }
        y_local[i] = soma;
    }

    MPI_Gather(y_local, linhas_locais, MPI_DOUBLE,
               y, linhas_locais, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    if (processo == 0) {
        double tempo = MPI_Wtime() - inicio;
        printf("M=%d N=%d processos=%d tempo=%.6f s y[0]=%.0f y[%d]=%.0f\n",
               m, n, processos, tempo, y[0], m - 1, y[m - 1]);
    }

    free(a);
    free(y);
    free(x);
    free(a_local);
    free(y_local);
    MPI_Finalize();
    return 0;
}
```